



Notes Management System



Project Overview

The Notes Management System is a full-stack web application developed using Flask (Python) and SQLite3.

It allows users to securely create, manage, and organise their personal notes with full CRUD functionality.

Each user has a private workspace, ensuring data security and isolation.



Abstract

This application enables users to:

- Register and log in securely
- Create, view, update, and delete notes
- Pin important notes to the top
- Sort notes by newest or oldest
- Manage a personal profile with photo upload
- Search notes by title or content
- Reset forgotten passwords via username
- Send contact messages directly from the app

It includes authentication, session management, password encryption, profile photo storage, and database integration, making it a complete and modern web solution.



Objectives

- Build a user-friendly notes management platform
- Implement secure authentication using Flask sessions
- Perform CRUD operations on notes
- Ensure user-specific data privacy
- Integrate Flask with SQLite3 database
- Allow users to pin and prioritise important notes
- Enable profile customisation with photo upload
- Provide date-based note sorting



Tech Stack

Layer	Technology	Purpose
Frontend	HTML5, CSS3, Bootstrap 5	UI Design & Responsiveness
Backend	Flask (Python)	Application Logic & Routing
Database	SQLite3	Data Storage
Security	Werkzeug	Password Hashing

Layer	Technology	Purpose
Email	Flask-Mail (Gmail SMTP)	Contact & Registration Mail
File Store	static/uploads folder	Profile Photo Storage
Server	Flask Development Server	Local Development & Testing

Installation & Setup

1. Clone the Repository

```
git clone https://github.com/your-username/notes-management-system.git
cd notes-management-system
```

2. Create Virtual Environment

```
python -m venv venv
venv\Scripts\activate    # Windows
```

3. Install Dependencies

```
pip install -r requirements.txt
```

4. Run the Application

```
python app.py
```

Requirements

```
blinker==1.9.0
click==8.4.1
colorama==0.4.6
Flask==3.1.3
Flask-Mail==0.10.0
itsdangerous==2.2.0
Jinja2==3.1.6
MarkupSafe==3.0.3
sqlite3 (built-in)
```

```
Werkzeug==3.1.8
```

Project Modules

User Module

- Register new account
- Login with credentials
- Logout securely
- Session management using Flask

- Forgot password / reset via username
- Profile page with photo upload and edit

Notes Module (CRUD)

Operation	Route	Description
Create	/addnote	Add a new note
Read All	/viewall	View all user notes
Read One	/viewnotes/<id>	View single note
Update	/updatenote/<id>	Edit existing note
Delete	/deletenote/<id>	Delete note permanently
Pin/Unpin	/pinnote/<id>	Toggle pin status of a note
Search	/search	Search notes by title or content

Profile Module

Operation	Route	Description
View Profile	/profile	View username, email and photo
Edit Profile	/edit_profile	Update username, email and upload photo

Other Routes

Route	Method	Description
/about	GET	About page
/contact	GET/POST	Contact form — sends email via Gmail SMTP
/forgotpassword	GET/POST	Reset password by username

Database Structure

Users Table

Column	Type	Description
id	INT (PK)	Unique user ID
username	VARCHAR(100)	Unique username

Column	Type	Description
email	VARCHAR(100)	User email
password	VARCHAR(255)	Encrypted password (Werkzeug)
profile_pic	VARCHAR(200)	Filename of uploaded profile photo

Notes Table

Column	Type	Description
id	INT (PK)	Unique note ID
title	VARCHAR(100)	Note title
content	TEXT	Note content
created_at	TIMESTAMP	Creation time
user_id	INT (FK)	Linked to users.id
is_pinned	TINYINT(1)	1 = pinned, 0 = normal (default 0)

New Features Added

Pin Important Notes

- Any note can be pinned with one click from the notes list or single note view
- Pinned notes always appear at the top of the list in a highlighted section
- Clicking Pin/Unpin toggles the state instantly



Sort Notes by Date

- Sort dropdown in the top-right of the notes page
- Newest First (default) — most recent notes appear first
- Oldest First — oldest notes appear first
- Sorting is preserved via URL parameter (?sort=newest / ?sort=oldest)



Profile with Photo Upload

- Profile avatar appears in the navbar (round) with username
- Shows first initial if no photo is uploaded
- Clicking the avatar opens a dropdown with name, email, and menu links
- Edit Profile page allows updating username, email, and uploading a new photo
- Photos are stored in the static/uploads/ folder



Security Features

- Passwords hashed using generate_password_hash()

- User-specific note access — notes are private per user
- Session-based authentication
- Protected routes — login required for all note operations
- File upload validation (only image types allowed)
- ON DELETE CASCADE for database integrity



Frontend Features

- Responsive UI using Bootstrap 5
- Custom pastel purple-pink theme (style.css)
- Glassmorphism navbar with blur effect
- Animated cards with hover lift effect
- Profile avatar dropdown in navbar
- Pinned notes section with highlighted border
- Sort by date dropdown button
- Flash messages for user feedback



Application Workflow

1. User Registration

User signs up → password hashed → data stored in SQLite3 → redirected to login

2. Login

Credentials verified → session created (user_id, username, email, profile_pic) → redirected to notes

3. Notes Management

User creates, views, edits, deletes, pins, and searches their personal notes

4. Profile Management

User updates username/email and uploads a profile photo → shown in navbar instantly

5. Logout

Session cleared → redirected to login page



Learning Outcomes

- Build full-stack apps with Flask and SQLite3
- Understand CRUD operations
- Use Jinja2 templating
- Implement authentication and sessions
- Work with relational databases

- Handle file uploads in Flask
- Send emails using Flask-Mail and Gmail SMTP
- Build responsive UIs with Bootstrap 5 and custom CSS

Conclusion

This project demonstrates how to build a secure, feature-rich, database-driven web application using Flask. It is ideal for beginners learning Full Stack Python Development and covers authentication, CRUD, file uploads, email integration, and modern UI design.

Future Improvements

- Add categories / tags for notes
- Deploy on cloud (AWS / Render / Railway)
- Add REST API support
- Dark mode toggle
- Export notes as PDF
- Two-factor authentication (2FA)



Author

Gantedi Srinidhi



License

This project is open-source and available under the MIT License.